FULL-STACK REPOSITORY

# Structure & DX Audit

AstroGuide | Angular frontend + Python/FastAPI backend

**VERDICT: NEEDS IMPORTANT CLEANUP**

Sound project split; sanitize credential examples and untrack the venv.

## Executive summary

The repository has distinct backend/ and frontend/ directories, each with its own dependency definition. Angular has a matching npm lockfile. You already run the services in separate terminals; that is a clear and appropriate local development workflow. Docker Compose or a concurrent runner is optional, not a required fix.

Two findings merit prompt action: a backend virtual environment is committed to Git, and the tracked root .env.example contains values that do not look like safe placeholders. Do not copy or reuse those values. Rotate any credentials that were real; changing the file alone does not remove its earlier Git history.

## Priority actions

- HIGH - credentials: Replace values in the tracked .env.example with unmistakable placeholders. Rotate any real credentials and review Git history/deployment secrets.
- HIGH - repository hygiene: backend/.venv312 has 5,011 tracked files. Ignore it and remove it from the Git index; preserve local files until you confirm it is unused.
- MEDIUM - configuration: Complete the backend environment template, align its SMTP setting with code, and configure the production API URL and CORS origins.
- LOW - onboarding: Add a root README describing the existing two-terminal workflow. Orchestration remains optional.

## Scope and evidence

Inspected tracked paths, ignore rules, dependency manifests, environment variable names (not values), backend config/startup, Angular environment settings, and lockfile metadata. The local .env is ignored and was not tracked in the inspected index. No secret values are included here.

# Findings in detail

### 1. Tracked virtual environment - HIGH

The Git index contains 5,011 files under backend/.venv312. Committing a venv bloats clones and can include machine-specific binaries. A second backend environment, backend/.venv, is present locally and is ignored. Choose one working environment; do not remove either local directory until you know which one you use.

### 2. Unsafe tracked environment template - HIGH

The committed root .env.example contains database and JWT values that are not clearly placeholders. This audit cannot determine whether these are live credentials. Replace them with unmistakable placeholders, rotate any real values, and review Git history and deployment secrets. Keep .env.example committed only after sanitizing it; keep .env ignored.

### 3. Ignore strategy - MEDIUM

The root .gitignore covers several common Python, Angular, and secret files; frontend/.gitignore supplies Angular-specific rules. Root rules miss .venv312, pytest caches, and SQLite files. Consolidate common rules at the root; keeping frontend/.gitignore for Angular-only exceptions is optional.

### 4. Environment template mismatch - MEDIUM

Backend code reads GEOAPIFY_API_KEY, GEMINI_API_KEY, LLM_PROVIDER, OLLAMA_BASE_URL, OLLAMA_MODEL, and GEMINI_VISION_MODEL, among others, while the tracked template omits these. The template defines SMTP_TLS, but email_service.py checks SMTP_SSL and otherwise uses STARTTLS. Document settings actually consumed by the application.

### 5. Production frontend configuration - MEDIUM

Angular production environment currently points at 127.0.0.1:8000, which points to each end user?s own machine after deployment. Set the deployed API base URL in production build configuration and allow only deployed frontend origin(s) in backend CORS. Never place server API keys in Angular source.

### 6. Run workflow and dependency tracking - LOW

The two-terminal workflow is fine and matches how you already run the app. frontend/package-lock.json exists and its root name/version match package.json. backend/requirements.txt pins exact versions but is not hash-locked. No root README or one-command orchestration file was found. requirements.txt is UTF-16 encoded; local pip parsed it, but the offline dry-run could not complete because an unavailable package was not cached.

# Recommended layout

```
AstroGuide/
.gitignore
.env.example              # backend template; placeholders only
README.md                 # prerequisites and two-terminal run steps
backend/
.venv/                  # local Python environment; ignored
requirements.txt
alembic.ini, alembic/, app/, tests/
frontend/
package.json, package-lock.json, angular.json
src/environments/       # API URL only; no secrets
```

# Step-by-step remediation

### 1. Sanitize the tracked template and rotate credentials

Edit .env.example: replace every credential-like value with replace_me, list only backend variables, and correct the SMTP flag to SMTP_SSL=false (or document SSL explicitly). Add Geoapify, Gemini/LLM, database, JWT, and mail variables as applicable. Rotate any real credentials first. Do not put secrets in frontend source.

### 2. Replace root .gitignore with this shared baseline

```
# Secrets and local environment files
.env
.env.*
!.env.example
!.env.*.example

# Python
__pycache__/
*.py[cod]
*$py.class
.Python
.venv/
.venv*/
venv/
ENV/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
.coverage.*
htmlcov/

# Node and frontend build output
node_modules/
dist/
build/
.angular/
.next/
.nuxt/
coverage/
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# Local databases
*.db
*.sqlite
*.sqlite3

# Logs, temporary files, IDE and OS
*.log
tmp/
temp/
.idea/
.vscode/
.DS_Store
Thumbs.db
```

The frontend-specific ignore file may remain if it carries Angular-only exclusions; shared rules belong at the repository root.

### 3. Untrack the committed venv (local files are preserved)

```
git rm -r --cached --ignore-unmatch -- backend/.venv312
git add .gitignore .env.example
git status --short
git ls-files backend/.venv312
```

The last command should print no paths. The index removal does not delete the local venv. Remove that local directory only after confirming you no longer need it.

# Run, deploy & verify

### 4. Keep your existing two-terminal workflow

Terminal 1 - from the repository root (use backend/.venv if that is your working environment):

```
.\backend\.venv\Scripts\python.exe -m uvicorn app.main:app --app-dir backend --reload --host
  127.0.0.1 --port 8000
```

Terminal 2 - start Angular:

```
Set-Location frontend
npm ci
npm start
```

### 5. Update deployment settings and document setup

- Set Angular production API URL to the deployed backend address; the URL is configuration, not a secret.
- Set backend CORS allow_origins to the actual frontend URL(s); avoid wildcard origins when credentials are enabled.
- Add a root README with Python/Node versions, database/migrations, environment setup, the two-terminal commands, tests, and production configuration.
- Optionally generate a hash-locked backend requirements file if strict reproducibility is needed.

# Completion checklist

- .env.example contains placeholders only; any formerly valid credential is rotated.
- git check-ignore backend/.venv312 and git check-ignore frontend/node_modules both confirm ignored paths.
- git ls-files backend/.venv312 returns no output; no generated environment or build artifacts remain tracked.
- Frontend production API URL and backend CORS origins match deployment.
- From clean environments, npm ci and pip install -r backend/requirements.txt complete; then run the frontend build and backend tests.

**RECOMMENDATION**

**Sanitize credential handling and untrack the venv before the next push.**